

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	Furthose samreen B	Reg. No.:	192511164
Section / Batch:		Date:	27/08/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q10. Hamiltonian Path Problem (Backtracking / NP-Complete Problem)

Given a graph representing cities and connections, determine whether a path exists that visits each vertex exactly once. Clearly define constraints such as vertex visitation and path continuity. Analyze how these constraints influence the solution space. Develop a backtracking-based approach to explore all possible paths. Apply constraint checking to ensure valid paths. Justify your solution and discuss its computational complexity.

Q10. Hamiltonian Path Problem (Backtracking / NP-Complete Problem)

1. Problem Statement

The Hamiltonian Path Problem is to determine whether a graph contains a path that visits every vertex exactly once. A valid path must follow the edges of the graph without revisiting any vertex. Since this is an NP-Complete problem, a backtracking approach is commonly used to explore all possible paths.

2. Input

- A graph $G(V, E)$ where:
 - V = Set of vertices (cities)
 - E = Set of edges (connections)

3. Output

- **True** – A Hamiltonian Path exists.
- **False** – No Hamiltonian Path exists.

4. Constraints

- Every vertex must be visited **exactly once**.
- Consecutive vertices in the path must be connected by an edge.
- No vertex can be revisited.
- The path must contain all vertices.

5. Problem Analysis

The Hamiltonian Path problem has a very large search space because every vertex can be a possible starting point, and each step may have several choices. Invalid paths are discarded using constraint checking, while backtracking explores other possibilities until a valid path is found or all options are exhausted.

6. Backtracking Pseudocode

Algorithm HamiltonianPath(Graph G)

Input : Graph $G(V, E)$

Output: True if Hamiltonian Path exists, otherwise False

For each vertex v in G do

 Mark all vertices as unvisited

$Path[1] \leftarrow v$

 If Backtrack($v, 1$) = True then

 Return True

 End If

End For

Return False

Function Backtrack(Current, Count)

Mark Current as visited

If Count = TotalVertices then

 Print Path

 Return True

End If

For each Neighbor of Current do

 If Neighbor is not visited AND
 Edge(Current, Neighbor) exists then

 Path[Count+1] ← Neighbor

 If Backtrack(Neighbor, Count+1) = True then
 Return True
 End If

 End If

End For

Mark Current as unvisited

Return False

7. Algorithm

1. Read the graph as input.
2. Choose each vertex as the starting point.
3. Mark the current vertex as visited.
4. Visit every unvisited adjacent vertex recursively.
5. Check whether all vertices have been visited.
6. If yes, return **True**.
7. If no valid path is found, backtrack by unmarking the vertex.
8. Repeat until all possibilities are explored.
9. Return **False** if no Hamiltonian Path exists.

8. Constraint Checking

- Ensure every vertex is visited only once.
- Ensure movement is only through connected vertices.
- Reject paths containing repeated vertices.
- Accept the solution only when all vertices are included.

9. Solution Justification

Backtracking systematically explores every possible path while eliminating invalid paths early using constraint checking. It guarantees finding a Hamiltonian Path if one exists. Although computationally expensive, it is a complete and correct approach for solving NP-Complete problems.

10. Complexity Analysis

Time Complexity

- **O(N!)**
- The algorithm may examine all possible vertex arrangements in the worst case.

Space Complexity

- **O(N)**
- Stores the visited array, recursion stack, and current path.

11. Advantages

- Finds the correct solution if one exists.
- Simple recursive implementation.
- Suitable for small and medium-sized graphs.
- Uses pruning to avoid invalid paths.

12. Disadvantages

- High execution time for large graphs.

- Not suitable for dense graphs with many vertices.
- Exponential search space.

13. Applications

- Route planning and navigation.
- Network design.
- Robot path planning.
- Circuit board design.
- DNA sequencing.
- Logistics and transportation.
- Scheduling problems.

14. Conclusion

The Hamiltonian Path Problem determines whether every vertex in a graph can be visited exactly once. A backtracking algorithm recursively explores all possible paths while checking constraints such as vertex visitation and edge connectivity. The algorithm guarantees correctness but has a worst-case time complexity of $O(N!)$, making it practical mainly for smaller graphs.